

NIVELL 1**Exercici 1**

El Country Manager d'Alemanya necessita revisar urgentment les transaccions del dia 12 de març de 2022.

Escenari:

El quadre de comandament financer tarda massa a carregar i el cost és inacceptablement alt. S'ha identificat que el problema crític són els informes que creuen vendes amb regions geogràfiques.

Tasca:

1. Escriu la consulta que uneix (JOIN) transaccions i companyies.
2. Filtra els resultats per la data indicada i el país "Germany".
3. Sense executar la consulta, realitza un "Dry Run" (auditoria de costos). Observació: Fixa't que BigQuery llegeix gairebé tota la taula tot i demanar només un dia (Full Table Scan).

Resposta:

A partir del DataSet silver que es el lloc on tenim les dates mes netes creem el següent codi que en executar-se tindria un cost de 5.91MB. Si tenim en compte que, aquesta consulta la farien varies vegades al dia diferents departaments, implica un cost elevat.



```
1 SELECT
2   c.country,
3   DATE(t.timestamp),
4   t.transaction_id,
5   t.amount,
6 FROM `sprint3_silver.transactions_clean` AS t
7 INNER JOIN `sprint3_silver.companies_clean` AS c
8   ON t.business_id = c.company_id
9 WHERE c.country = 'Germany' AND
10  DATE(t.timestamp) = '2022-03-12'
11 ORDER BY t.amount DESC;
```

✓ Esta consulta procesará 5.91 MB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Exercici 2

Escenari:

Després del diagnòstic de l'exercici anterior, hem confirmat que filtrar la taula de transaccions per data o per empresa provoca un Full Table Scan (lectura completa de la taula), cosa que és ineficient. Per solucionar-ho a la capa de consum (Gold), materialitzarem una versió optimitzada de la taula de fets.

Problema del sandbox: com que utilitzem un entorn gratuït sense targeta de crèdit, BigQuery esborra automàticament les dades de les particions amb més de 60 dies d'antiguitat. Atès que les nostres dades històriques són antigues, si particionem directament la taula, aquesta acabarà quedant buida. Per evitar-ho, ho resolrem en dos passos: primer, actualitzarem automàticament les dates de les dades i, després, aplicarem l'optimització física.

Tasca:

Pas 1: Generació de Dades Recents (Mocking Data)

Crea una taula intermèdia anomenada `sprint3_silver.transactions_recent` a partir de la taula `sprint3_silver.transactions_clean`. El teu objectiu és mantenir totes les columnes, però substituir el timestamp original per un de nou, generat aleatòriament perquè caigui dins dels últims 50 dies.

Pista Tècnica: Per fer-ho sense escriure totes les columnes a mà, pots utilitzar `SELECT * EXCEPT(timestamp)`. Per calcular la data aleatòria, hauràs de restar una quantitat de dies a l'instant actual.

Et seran molt útils les funcions `TIMESTAMP_SUB()`, `CURRENT_TIMESTAMP()`, i la combinació `CAST(RAND() * 50 AS INT64)` per generar els dies de desfasament.

Resposta

Seguint les indicacions, creem el codi per la taula intermèdia amb dades aleatòries. Aquesta consulta crea la taula a partir de `transactions_clean`, mantenint totes les columnes originals però substituint el timestamp original per un generat artificialment dins dels últims 50 dies. Hem utilitzat `EXCEPT(timestamp)` que seleccionarà totes les columnes originals excepte aquesta evitant haver de llistar manualment cada columna. La funció `CAST(RAND()*50 AS INT64)` genera un número decimal aleatori entre 0 i 1 (`RAND()`), el multiplica per 50 per obtenir un valor entre 0 i 50, i el converteix a enter (`INT64`) per poder-lo usar com a interval de dies. `TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL ... DAY)` resta el nombre de dies aleatori al moment actual, generant un timestamp nou que sempre caurà dins de la finestra dels últims 50 dies. La taula es idèntica a l'anterior però els timestamps més recents i aleatoris:

```
1 CREATE OR REPLACE TABLE proverbial-deck-498507-c9.sprint3_silver.transactions_recent AS
2 SELECT * EXCEPT(timestamp),
3     TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL CAST(RAND() * 50 AS INT64) DAY)
4     AS timestamp
5 FROM
6     proverbial-deck-498507-c9.sprint3_silver.transactions_clean;
```

✓ Esta consulta procesará 11.09 MB cuando se ejecute.

Podem veure a continuació el resultat de l'execució del codi i la vista de a nova taula creada:

```

1 CREATE OR REPLACE TABLE proverbial-deck-498507-c9.sprint3_silver.transactions_recent AS
2 SELECT * EXCEPT(timestamp)
3   FROM proverbial-deck-498507-c9.sprint3_silver.transactions_clean;
4 AS timestamp
5 FROM
6   proverbial-deck-498507-c9.sprint3_silver.transactions_clean;
7
8
9
10
11
12
13
14
15
16
17
18

```

Se completó la consulta
Se está usando la cuota de procesamiento según demanda

Recientes 6:07 p.m. Resultados de la consulta Chat View all jobs

6:07 p.m. CREATE OR REPLAC...
10:34 a.m. SELECT c company...
8:02 a.m. CREATE OR REPLAC...

Si necesitas ayuda para optimizar o depurar tu consulta, revisa la documentación. [Obtén más información](#)

Tiempo transcurrido 2.23 s
Tiempo de ranura consumido 1.97 s
Bytes generados y movidos 27.24 MB

proverbial-deck-498507-c9 / Datasets / sprint3_silver / Tables / transactions_recent

transactions_recent Consulta Chat Abrir en Compartir Copiar Instantánea Borrar Exportación

Esquema Detalles Vista previa Explorador de tablas Vista previa Estadísticas Linaje Perfil de datos Calidad de los datos

Fila	transaction_id	amount	lat	longitude	product_ids	user_id	card_id	business_id	declined	timestamp
1	78C7704EF5D1-460C-95BF-DB...	294.36	40.716481533403559	-74.0119302...	99	1	CcU-4856	b-2334	0	2026-05-03 16:07:54.694534 UTC
					80					
					34					
2	7E438CE2-B3E0-4C8C-9EA7-0D...	554.76	40.71392571663835	-73.9981356...	67	1	CcU-4856	b-2530	0	2026-06-04 16:07:54.694534 UTC
					20					
					23					
					21					
3	00DBF1C6-FF88-4E47-B26B-BC...	272.45	40.71746049737034	-73.9960221...	81	1	CcU-4856	b-2314	0	2026-04-21 16:07:54.694534 UTC

Pas 2: Creació de la Taula Optimitzada (Partitioning & Clustering)

Ara, crea la teva taula definitiva `sprint3_gold.fact_transactions_optimized` a partir de les dades recents que acabes de generar a `transactions_recent`. Has de construir la sentència DDL per configurar la taula amb les següents estratègies físiques d'emmagatzematge:

- **Particionament (Partitioning):** Divideix la taula per la data del camp `DATE(timestamp)`. Això permetrà que les consultes que filtrin per dia (`WHERE date = ...`) només llegeixin la partició necessària.
- **Clustering:** Ordena les dades dins de cada partició per `business_id`. Això accelerarà dràsticament els filtres per companyia i els encreuaments (`JOINS`) amb la dimensió d'empreses.

Resposta

Hem particionat i claustritzat la taula optimitzada per tal d'accelerar la lectura i el rendiment de les nostres consultes amb les funcions:

PARTITION BY DATE : divideix la taula per data .

CLUSTER BY business_id: dins de cada partició, ordena i agrupa les files per `business_id`.

Aquesta taula no modifica les dades però si canvia totalment com s'emmagatzemen, fent-la més optima i reduint els costos ja que, BigQuery sap on guarda les dades i no ha de llegir ara total la taula.

Es mostra el codi a continuació i els detalls de resultat de la taula:

```

1 CREATE OR REPLACE TABLE proverbial-deck-498507-c9.sprint3_gold.fact_transactions_optimized
2 PARTITION BY DATE(timestamp)
3 CLUSTER BY business_id
4 AS SELECT * FROM
5 proverbial-deck-498507-c9.sprint3_silver.transactions_recent;
6
7
8
9
10
11
12
13
14
15
16
17
18

```

Esta consulta procesará 11.85 MB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Recientes < Resultados de la consulta

6:07 p.m. Información del trabajo Resultados Detalles de la ejecución

✓ CREATE OR REPLAC... Esta declaración creó una nueva tabla con el nombre transactions_recent.

10:34 a.m.

ID de la tabla	proverbial-deck-498507-c9.sprint3_gold.fact_transactions_optimized
Creado	10 jun 2026, 11:18:05 a.m. UTC+2
Última modificación	10 jun 2026, 11:18:05 a.m. UTC+2
Vencimiento de la tabla	NUNCA
Ubicación de los datos	EU
Intercalación predeterminada	
Modo de redondeo predeterminado	ROUNDING_MODE_UNSPECIFIED
No distingue mayúsculas de minúsculas	false
Descripción	
Etiquetas	
Claves primarias	
Etiquetas	
Tipo de tabla	Particionada
Particionada por	DAY
Particionada en el campo	timestamp
Vencimiento de la partición	60 días
Filtro de partición	No es obligatorio
Agrupada por	business_id

Exercici 3

Escenari:

Escenari: Ja has creat la taula `sprint3_gold.fact_transactions_optimized` amb les estratègies de Particionament i Clustering.

Ara toca demostrar a l'equip la millora de rendiment i l'estalvi de costos que has aconseguit. Simularem una petició típica de negoci: "Volem veure totes les transaccions dels últims 30 dies".

Tasca:

L'objectiu és clar i directe: aplicar exactament la mateixa consulta a dues taules diferents i comparar-ne el cost computacional.

Construeix una consulta SQL que seleccioni totes les columnes (`SELECT *`) i filtri les dades dels últims 30 dies.

NO executis les consultes encara. En aquest exercici només utilitzarem el validador de BigQuery ("Dry Run", a dalt a la dreta de l'editor) per visualitzar els Bytes processats de manera gratuïta.

Fes la comparativa seguint aquests dos passos:

- Pas 1 (Taula no optimitzada): Apunta la teva consulta a la taula `sprint3_silver.transactions_recent`. Observa el validador i fes una captura de pantalla dels bytes que processaria.
- Pas 2 (Taula optimitzada): Sense canviar res de la lògica de la teva consulta, modifica només el `FROM` perquè ara apunti a la taula particionada `sprint3_gold.fact_transactions_optimized`. Observa com canvia el validador i fes la segona captura.

Resposta

Codi filtrant per data a 30 dies per ambdues taules. Es mostra a sota de cada captura els bytes que consumeix cada consulta:

```
1 SELECT
2 * FROM
3 proverbial-deck-498507-c9.sprint3_silver.transactions_recent
4 WHERE
5 DATE(timestamp)>=DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)
6
```

Esta consulta procesará 11.85 MB cuando se ejecute.

```
1 SELECT
2 * FROM
3 proverbial-deck-498507-c9.sprint3_gold.fact_transactions_optimized
4 WHERE
5 DATE(timestamp)>=DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)
6
```

Esta consulta procesará 7.02 MB cuando se ejecute.

Tal i com es pot veure a les captures, amb la taula sense optimitzar surten 11.85MB i amb la optimitzada 7,02MB això representa un **40,76%** de millora amb la estratègia aplicada. BigQuery s'estalvia llegir gairebé un 41% de dades a la mateixa taula per a trobar la informació, gràcies al particionament per data. D'aquesta manera, es redueixen significativament el consum de recursos.

Exercici 4

Escenari:

El Director General té un quadre de comandament que mostra les "Vendes Totals per Dia". Ell (i 50 managers més) refresquen aquest gràfic constantment.

Problema:

Si utilitzes una consulta normal o una Vista estàndard, cada vegada que algú refresca la pàgina, BigQuery recalcula la suma de milions de files. És lent i estàs pagant pel mateix càlcul una vegada i una altra.

Solució:

Utilitzar una Vista Materialitzada. Aquesta tecnologia guarda el resultat del càlcul a la memòria cau i només l'actualitza si entren dades noves (incremental).

Tasca:

Crea una Vista Materialitzada anomenada `sprint3_gold.mv_daily_sales` que mostri les Vendes Totals per Dia.

Resposta

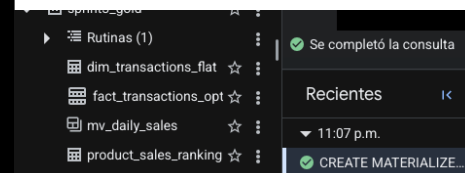
Hem creat una Vista Materialitzada que guarda el resultat precalculat de les vendes diàries i només es torna a calcular quan entren dades noves de forma incremental. Això és possible gràcies a la configuració `OPTIONS (enable_refresh = true)` que permet a BigQuery detectar automàticament quan arriben transaccions noves i actualitzar únicament els nous registres, sense recalculer tot des de zero. Això redueix dràsticament el cost de BigQuery i ofereix respostes immediates al dashboard als 50+ managers que el consulten diàriament, impactant de forma positiva en l'eficiència operativa i financera de l'empresa.

A continuació es mostra el codi:

```
Archivo  Editar  Ejecutar  Herramientas

1 CREATE MATERIALIZED VIEW proverbial-deck-498507-c9.sprint3_gold.mv_daily_sales
2 OPTIONS (
3   enable_refresh = true
4 )
5 AS
6 SELECT
7   DATE(t.timestamp) AS date_of_sales,
8   SUM(t.amount) AS total_sales,
9   COUNT(t.transaction_id) AS num_transactions
10 FROM
11   proverbial-deck-498507-c9.sprint3_gold.fact_transactions_optimized AS t
12 WHERE t.declined = 0
13 GROUP BY DATE(t.timestamp);
14

✓ Esta consulta procesará 0 B cuando se ejecute.
```



Afegim el (ROUND, 2), a una altre consulta independent ja que, a la mateixa no ens ho permet, no accepta arrodonir el resultat abans de fer la suma total, ja que entèn que això pot aletrar el resultat.

```

1 SELECT
2   date_of_sales,
3   ROUND(total_sales, 2) AS total_sales,
4   num_transactions
5 FROM sprint3_gold.mv_daily_sales
6

```

Se completó la consulta

Aquesta serà la consulta que faràn servir els mànagers diariament. No ens sortirà cap transacció amb el filtre CURRENT DATE ja que, es van modificar les dades fins al dia 09.06.

Consulta sin título Ejecutar Descargar

```

1 SELECT *
2 FROM proverbial-deck-498507-c9.sprint3_gold.mv_daily_sales
3 WHERE date_of_sales = CURRENT_DATE();

```

Esta consulta procesará 1.2 KB cuando se ejecute.

Executem la consulta amb el filtre ORDER BY date_of_sales per veure les últimes transaccions de mes recent a mes nova per comprovar que funciona correctament:

Fila	date_of_sales	total_sales	num_transactions
1	2026-06-09	247011.46	984
2	2026-04-20	256141.01	1004
3	2026-04-24	493138.45	1915
4	2026-05-04	496666.82	1916
5	2026-05-23	503071.55	1924
6	2026-05-13	508185.42	1932
7	2026-05-11	497098.04	1938
8	2026-05-03	493999.06	1944
9	2026-05-30	503024.52	1948

NIVELL 2

Exercici 1

Escenari:

La direcció comença a fer preguntes que no es resolen amb un simple GROUP BY. Necessitem anàlisi de tendències i comportament d'usuari.

L'equip de Màrqueting vol analitzar el comportament dels nostres millors clients per dissenyar l'estratègia del pròxim any. Definim "VIP" com aquells amb una despesa acumulada superior a 500€. Necessiten un informe que, per a cada VIP, mostri el seu nom, contacte i el seu patró de compra: quantes vegades ha comprat, quant gasta de mitjana i quina va ser la seva compra rècord.

Tasca:

1. Crea una CTE anomenada VIP_Stats que agrupi per usuari i calculi:
 - La Despesa Total (SUM).
 - La Quantitat de Transaccions (COUNT).
 - El Tiquet Mitjà (AVG), arrodonit a 2 decimals.
 - La Compra Màxima (MAX).
 - Filtre: Mantén només aquells la Despesa Total dels quals sigui > 500.
2. Creua la CTE amb users_combined per obtenir les dades personals. Requisits de Sortida:
 - Columnes: user_id, nom_complet, email, num_compres, tiquet_mig, max_compra, total_gastat.
 - Ordenat per total_gastat descendent.

Resposta

Hem creat la CTE amb el codi que es mostra a la captura per al departament de Màrqueting amb les dades corresponents per poder analitzar amb èxit el comportament dels clients i categoritzarlos com a VIP.

Aquesta consulta utilitza un **CTE (Common Table Expression)** anomenat VIP_Stats per calcular, en un primer pas, les mètriques agregades de cada usuari: el total gastat (SUM(amount)), el nombre de transaccions úniques (COUNT(DISTINCT transaction_id)), el tiquet mitjà (AVG(amount)) i la compra màxima (MAX(amount)). El filtre HAVING total_amount > 500 s'aplica després de l'agregació per quedar-se només amb els usuaris que han gastat més de 500€ en total.

En un segon pas, el resultat del CTE es creua amb la taula users_combined mitjançant un JOIN per enriquir les dades amb el nom i email de cada client, i s'ordena per total_amount DESC per visualitzar primer els clients de major valor.

Aquesta CTE ens permet segmentar als clients i poder identificar-los i categoritzar-los sense necessitat d'analitzar tota la base de dades.

```

WITH VIP_Stats AS (
  SELECT
    t.user_id,
    ROUND(SUM(t.amount), 2) AS total_amount,
    COUNT(DISTINCT t.transaction_id) AS num_transactions,
    ROUND(AVG(t.amount), 2) AS ticket_average,
    MAX(t.amount) AS max_amount
  FROM proverbial-deck-498507-c9.sprint3_gold.fact_transactions_optimized AS t
  GROUP BY t.user_id
  HAVING total_amount > 500
)

SELECT
  u.user_id,
  CONCAT(u.name, ' ', u.surname) AS name,
  u.email,
  v.total_amount,
  v.max_amount,
  v.ticket_average,
  v.num_transactions
FROM VIP_stats AS v
JOIN proverbial-deck-498507-c9.sprint3_silver.users_combined AS u
  ON v.user_id = u.user_id
ORDER BY total_amount DESC;

```

Resultat de l'execució amb la mostra dels clients VIP:

✓ Esta consulta procesará 5.41 MB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Recientes

▼ 12:59 p.m.

✓ WITH VIP_Stats AS (SELEC...

▼ 12:58 p.m.

✓ WITH VIP_Stats AS (SELEC...

▼ 6:53 p.m.

✓ SELECT * FROM proverbial...

▼ 6:50 p.m.

✓ SELECT * FROM proverbial...

▼ 6:22 p.m.

Resultados de la consulta

Chat

View all jobs

Guardar los resultados ▼

Abrir en ▼

Información del trabajo

Resultados

Visualización

JSON

Detalles de la ejecución

Gráfico de ejecución

Fila	user_id	name	email	total_amount	max_amount	ticket_average	num_transactions
1	289	Dxwgi Hwcru	dxwgi.hwcru@example.com	41486.54	554.71	441.35	94
2	318	Bnyr Astuw	bnyr.astuw@example.com	40348.5	615.51	443.39	91
3	455	Fwfil Phylgnmhnd	fwfil.phylgnmhnd@example.com	34491.27	724.44	522.6	66
4	417	Fqatil Rdwcjcvoa	fqatil.rdwcjcvoa@example.com	32016.11	675.78	410.46	78
5	185	Molly Gilliam	donec@outlook.couk	30333.03	659.31	275.75	110
6	285	Brcxprnj Lnpna	brcxprnj.lnpna@example.com	29249.51	630.52	417.85	70
7	193	Minerva Wilkins	aliquam.adipiscing@yahoo.edu	25485.05	646.0	380.37	67

Exercici 2

Escenari:

La Direcció Financera vol monitoritzar la "velocitat de vendes" diària. Necessiten un informe que compari el rendiment de cada dia contra el dia anterior per detectar caigudes brusques o pics de creixement (Day-over-Day Growth).

Estratègia d'Optimització:

Per a aquest càlcul NO has d'anar a la taula de fets original ni recalculer sumes. Com a bon Data Engineer, has d'utilitzar la Vista Materialitzada `sprint3_gold.mv_daily_sales` que vas crear a l'exercici anterior. Això garanteix que l'anàlisi de tendències sigui instantània i no consumeixi recursos de còmput innecessaris.

Tasca:

Crea una consulta sobre la vista materialitzada que generi un informe amb les següents 4 columnes:

- 1.Data: La data de venda.
- 2.Vendes_Avui: El total venut aquell dia
- 3.Vendes_Ahir: El total venut el dia anterior (usant funcions de finestra).
- 4.Diff_Percentual: El percentatge de creixement o decreixement respecte al dia anterior, arrodonit a 2 decimals.

Pista Tècnica:

Utilitza la funció `LAG(camp) OVER (ORDER BY data)` per llegir el valor de la fila anterior sense necessitat de fer JOINS complexos.

Resposta

Es mostra a continuació el codi per a poder realitzar la monitorització dels creixements o decreixements de vendes i la diferència en percentatge. Per realitzar-ho fem servir funcions de finestra a la vista materialitzada que ja ens fa l'actualització diària de vendes utilitzant una vista precalculada. D'aquesta manera aconseguim obtenir els càlculs d'una forma ràpida i optimitzada.

S'utilitza la funció de finestra `LAG(total_sales) OVER (ORDER BY date_of_sales)` per accedir al valor de vendes del dia anterior sense necessitat de fer cap JOIN, simplement "mirant enrere" una fila en l'ordre cronològic. El creixement percentual es calcula amb la fórmula $((\text{avui} - \text{ahir}) / \text{ahir}) * 100$, afegim el `NULLIF(..., 0)` per evitar errors en cas que el dia anterior tingués vendes zero, i arrodonint el resultat a 2 decimals amb `ROUND`.

Consulta sin título Ejecutar Descargar Compartir

```

1 SELECT
2   date_of_sales,
3   ROUND(total_sales, 2) AS today_sales,
4   ROUND(LAG(total_sales) OVER (ORDER BY date_of_sales), 2) AS yesterday_sales,
5   ROUND(
6     (total_sales - LAG(total_sales) OVER (ORDER BY date_of_sales))
7     / NULLIF(LAG(total_sales) OVER (ORDER BY date_of_sales), 0) * 100
8     , 2) AS Diff_percentual
9 FROM proverbial-deck-498507-c9.sprint3_gold.mv_daily_sales
10 ORDER BY date_of_sales ASC;
11
12

```

✓ Esta consulta procesará 816 B cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Es mostren els resultats del codi. La primera fila sempre sortirà null ja que no té cap anterior per a que la funció LAG trobi una fila.

Ja podem veure la diferencia percentual d'un dia a l'altre i poder prendre decisions basades en dades:

```

1 SELECT
2   date_of_sales,
3   ROUND(total_sales, 2) AS today_sales,
4   ROUND(LAG(total_sales) OVER (ORDER BY date_of_sales), 2) AS yesterday_sales,
5   ROUND(
6     (total_sales - LAG(total_sales) OVER (ORDER BY date_of_sales))
7     / NULLIF(LAG(total_sales) OVER (ORDER BY date_of_sales), 0) * 100
8     , 2) AS Diff_percentual
9 FROM proverbial-deck-498507-c9.sprint3_gold.mv_daily_sales
10 ORDER BY date_of_sales ASC;

```

✓ Esta consulta procesará 816 B cuando se ejecute.

Se está usando la cuota de procesamiento según demanda Ubicación de procesamiento: EU X

Recientes	Resultados de la consulta				
▼ 10:34 p.m.	Información del trabajo	Resultados	Visualización	JSON	Deta
✓ SELECT date_of_sales, ...	Fila	date_of_sales	today_sales	yesterday_sales	Diff_percentual
	1	2026-04-20	256141.01	null	null
	2	2026-04-21	507365.03	256141.01	98.08
	3	2026-04-22	507437.2	507365.03	0.01
	4	2026-04-23	532341.66	507437.2	4.91
	5	2026-04-24	493138.45	532341.66	-7.36
	6	2026-04-25	494475.54	493138.45	0.27
	7	2026-04-26	498359.61	494475.54	0.79

Exercici 3

Escenari:

El Director Financer (CFO) necessita visualitzar la corba de creixement anual. No li interessa tant el que es va vendre avui, sinó quant portem facturat en total des de l'inici de l'any fins avui (YTD - Year To Date).

Tasca:

Utilitzant la vista materialitzada mv_daily_sales (per no recalculer agregacions base), genera un informe amb tres columnes:

1.Data.

2.Vendes del Dia: Arrodonit a 2 decimals.

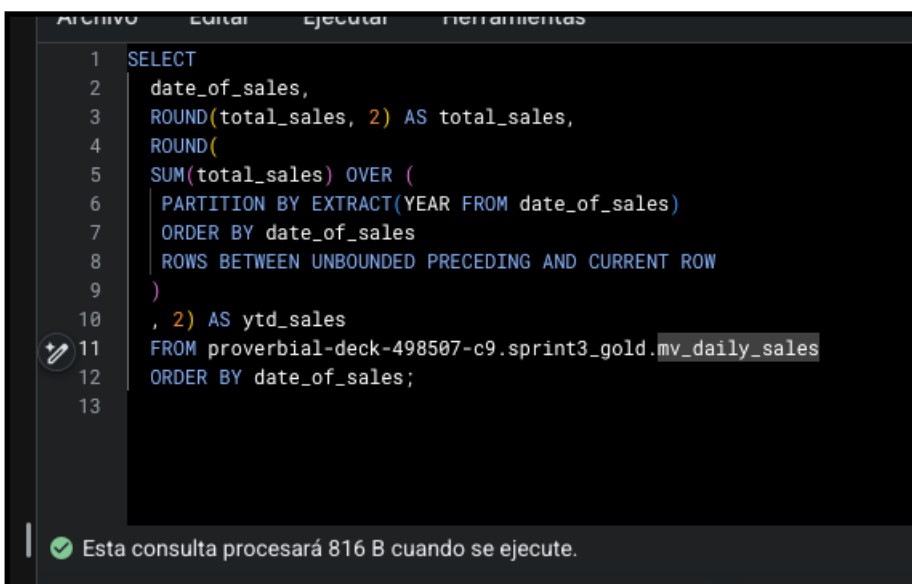
3.Vendes Acumulades YTD: Una suma progressiva de les vendes que ha de reiniciar-se cada 1 de Gener. El resultat final ha d'estar arrodonit a 2 decimals.

Pista Tècnica:

Per aconseguir el reinici anual, necessites dividir la finestra usant PARTITION BY EXTRACT(YEAR FROM data). L'estructura completa seria: SUM(...) OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW).

Resposta

Aquesta consulta genera l'informe de creixement anual acumulat (YTD) sol·licitat pel Director Financer utilitzant la vista materialitzada per garantir la màxima velocitat de còmput. La clau del codi és la funció de finestra SUM() OVER, que va acumulant la facturació dia a dia com una "bola de neu". Per aconseguir que aquest comptador es reiniciï automàticament cada 1 de gener, s'aplica un PARTITION BY que talla les dades per anys, i es delimita el càlcul des de l'inici del període fins al dia actual amb la instrucció ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW.



```
1 SELECT
2   date_of_sales,
3   ROUND(total_sales, 2) AS total_sales,
4   ROUND(
5     SUM(total_sales) OVER (
6       PARTITION BY EXTRACT(YEAR FROM date_of_sales)
7       ORDER BY date_of_sales
8       ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
9     )
10    , 2) AS ytd_sales
11 FROM proverbial-deck-498507-c9.sprint3_gold.mv_daily_sales
12 ORDER BY date_of_sales;
```

Esta consulta procesará 816 B cuando se ejecute.

Presentació de la vista taula amb les columnes on es poder veure els resultats demanats:

✓ Se completó la consulta Se está usando la cuota de procesamiento según demanda Ubicación de procesamiento: EU					
Recientes	Resultados de la consulta				
▼ 10:57 a.m.	Información del trabajo	Resultados	Visualización	JS	
✓ SELECT date_of_sales, ...	Fila	date_of_sales	total_sales	ytd_sales	
▼ 10:54 a.m.	1	2026-04-20	256141.01	256141.01	
✓ SELECT date_of_sales, ...	2	2026-04-21	507365.03	763506.04	
▼ 10:45 a.m.	3	2026-04-22	507437.2	1270943.24	
✓ SELECT date_of_sales, ...	4	2026-04-23	532341.66	1803284.9	
▼ 10:34 p.m.	5	2026-04-24	493138.45	2296423.35	
✓ SELECT date_of_sales, ...	6	2026-04-25	494475.54	2790898.89	
▼ 10:34 p.m.	7	2026-04-26	498359.61	3289258.5	
✓ SELECT date_of_sales, ...	8	2026-04-27	513028.89	3802287.39	

Exercici 4

Escenari:

L'equip de Màrqueting llança la campanya "A la tercera va la vençuda". Volen enviar un regal als usuaris que hagin completat la seva tercera transacció. No obstant això, el pressupost del regal no és fix: depèn de la "qualitat" del client en els seus inicis. Necessiten saber el Tiquet Mitjà d'aquestes 3 primeres compres per segmentar si envien un detall bàsic o un regal premium.

Tasca:

Genera un llistat dels usuaris que han arribat (o superat) la seva tercera compra, mostrant:

- 1.Dades d'usuari (user_id, nom_complet, email).
- 2.La data i l'import exacte de la 3a compra.
- 3.Mitjana de les 3 primeres: La mitjana de despesa de les seves transaccions 1, 2 i 3.

Pista Tècnica:

- Has d'utilitzar la funció ROW_NUMBER() per establir l'ordre cronològic de les compres.
- Has d'utilitzar la clàusula QUALIFY per filtrar.
- Repte d'Optimització: Busca l'estratègia de menor cost computacional. Tingues en compte que calcular .

Resposta

Aquesta consulta identifica els usuaris que han fet tres o més compres i mostra les seves dades junt amb la informació de la tercera transacció (data i import). A més, calcula la mitjana de despesa de les seves 3 primeres compres, que serveix per valorar la "qualitat" inicial del client.

Per fer-ho, numerem les compres de cada usuari per ordre cronològic mitjançant ROW_NUMBER() i, gràcies a QUALIFY row_num = 3, ens quedem únicament amb la fila corresponent a la tercera compra, exclouent automàticament tots els usuaris amb menys de tres transaccions. La mitjana de les tres primeres compres es calcula amb AVG() com a funció de finestra, afegim ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW per indicar-li que ha de considerar des de la primera compra de l'usuari fins a la fila actual, és a dir, exactament les compres 1, 2 i 3. QUALIFY filtra precisament la fila 3, aquest càlcul retorna sempre la mitjana de les tres primeres compres i no més. Finalment, fem un JOIN amb users_combined per obtenir les dades del client.

El resultat s'ordena de major a menor mitjana, de manera que l'equip de Màrqueting pot identificar ràpidament quins clients tenen més potencial i decidir si els envia un regal bàsic o premium.

```

1 SELECT
2   u.user_id,
3   CONCAT(u.name, ' ', u.surname) AS name,
4   u.email,
5   t.timestamp AS date_of_sale,
6   t.amount,
7   ROUND(
8     AVG(t.amount) OVER (
9       PARTITION BY t.user_id
10      ORDER BY t.timestamp
11      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
12    )
13   , 2) AS avg_3_sales,
14
15   ROW_NUMBER() OVER (PARTITION BY t.user_id ORDER BY t.timestamp) AS row_num
16
17 FROM proverbial-deck-498507-c9.sprint3_silver.transactions_clean AS t
18 INNER JOIN proverbial-deck-498507-c9.sprint3_silver.users_combined AS u
19   ON t.user_id = u.user_id
20 WHERE t.declined = 0
21 QUALIFY row_num = 3
22 ORDER BY avg_3_sales DESC;
23

```

✓ Esta consulta procesará 3.31 MB cuando se ejecute.

Mostra de resultats en executar el codi:

Recientes	IK	Resultados de la consulta							Chat	View all jobs
▼ 12:31 p.m.		Información del trabajo		Resultados	Visualización	JSON	Detalles de la ejecución		Gráfico de ejecución	
✓ SELECT u.user_id, CON...		Fila	user_id	name	email	date_of_sale	amount	avg_3_sales	row_num	
▼ 12:22 p.m.		1	444	Ekif Srqdlxkk	ekif.srqdlxkk@example.com	2016-05-12 20:56:06 UTC	674.66	581.87	3	
✓ SELECT u.user_id, CON...		2	2124	Ltbjxm ZieIgna	ltbjxm.zieIgna@example.com	2015-08-08 17:26:06 UTC	509.37	565.5	3	
▼ 10:57 a.m.		3	2762	Mykbqx Iodwgbk	mykbqx.iowdgbk@example.com	2017-06-29 09:05:13 UTC	695.08	545.52	3	
✓ SELECT date_of_sales, ...		4	3714	Vfhdkr Vfzkeju	vfhdkr.vfazkeju@example.com	2015-12-15 04:50:57 UTC	685.36	525.5	3	
▼ 10:54 a.m.		5	300	Aiet Jwcmzsbs	aiet.jwcmzsbs@example.com	2015-11-26 01:22:03 UTC	444.5	523.76	3	
✓ SELECT date_of_sales, ...		6	235	Medge Nieves	ac@icloud.couk	2015-04-28 15:30:22 UTC	499.17	522.82	3	
▼ 10:45 a.m.		7	3338	Hlyhem Awgcsld	hlyhem.awgcsld@example.com	2016-03-15 18:33:50 UTC	462.2	521.86	3	
✓ SELECT date_of_sales, ...		8	479	Qirpsrzg Canoxoykp	qirpsrzg.canoxoykp@example.c...	2015-07-20 07:38:47 UTC	301.34	517.92	3	
▼ 10:34 p.m.		9	456	Ysgrczww Xrfcht	ysgrczww.xrfcht@example.com	2015-07-10 13:34:28 UTC	561.31	517.54	3	
✓ SELECT date_of_sales, ...		10	3302	Wvnsow Yfbnpmru	wvnsow.yfbnpmru@example.co...	2017-10-14 08:30:48 UTC	426.37	514.93	3	
✓ SELECT date_of_sales, ...		11	3258	Dxcvsz Fqycuoma	dxcvsz.fqycuoma@example.com	2015-04-05 12:56:12 UTC	452.03	513.59	3	

NIVELL 3**Exercici 1****Escenari:**

Màrqueting necessita urgentment un Rànquing de Productes més Venuts.

Escenari:

Actualment, a la taula de transaccions, els productes venuts estan "amagats" dins d'un Array (una llista d'IDs). L'equip de vendes vol analitzar quins productes específics es venen més, però no poden fer-ho si les dades estan comprimides en una llista. Necessiten una Taula Detallada (Flat Table) on cada fila representi un producte venut, encara que això signifiqui que l'ID de la transacció es repeteixi.

Tasca:

Crea la taula `dim_transactions_flat` desnormalitzant la informació. Has d'"explotar" l'array de productes i creuar-lo amb el catàleg mestre per obtenir els noms i preus individuals.

Passos Tècnics:

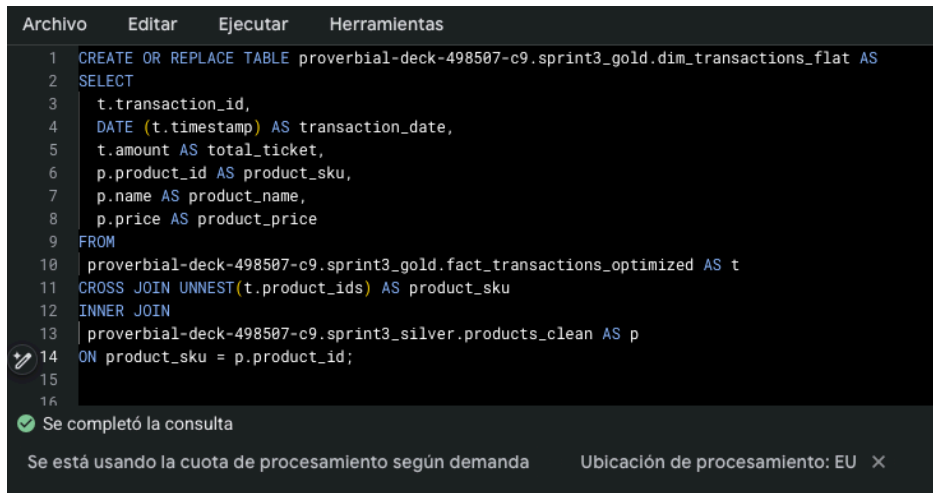
- Utilitza `CROSS JOIN UNNEST(product_ids)` per transformar l'Array en files individuals.
- Realitza un `JOIN` amb la taula `products` per obtenir el nom (`name`) i preu (`price`) de cada ítem.
Nota: Ves amb compte amb els tipus de dades en fer el `JOIN` (*Array de INT64 vs Product_ID STRING*).
- No agrupeu el resultat. Volem veure el desglossament línia per línia.

Resposta

Aquesta consulta crea la taula `dim_transactions_flat` desnormalitzant la taula de transaccions, on els productes venuts estaven "amagats" dins d'un array `product_ids`. L'objectiu és transformar aquesta estructura comprimida en una taula plana on cada fila representa un únic producte venut.

Per fer-ho s'utilitza `CROSS JOIN UNNEST(t.product_ids)` que "esclata" l'array de productes en files individuals, si una transacció té 5 productes, passa a ocupar 5 files, repetint les dades generals (`transaction_id`, `transaction_date`, `total_ticket`) a cadascuna. Posteriorment, un `INNER JOIN` amb `products_clean` creua cada `product_id` individual amb el catàleg de productes per obtenir el resultat amb el nom (`product_name`) i el preu (`product_price`) de cada un d'ells.

Aquesta estructura plana permet fer consultes simples com `GROUP BY product_name` per obtenir rànquings de productes, sense haver d'afegir més complexitat a la consulta.



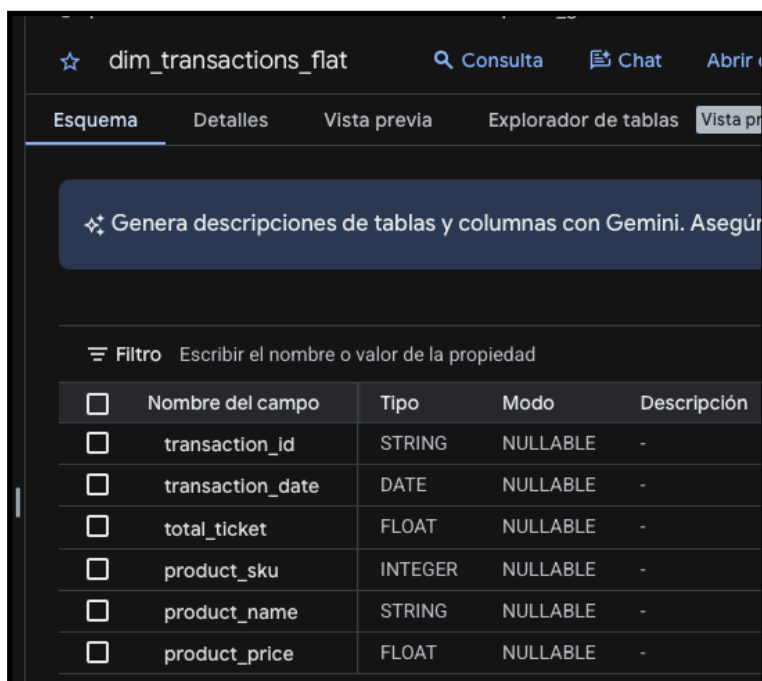
The screenshot shows a SQL query execution interface with a menu bar (Archivo, Editar, Ejecutar, Herramientas) and a query editor. The query is a CREATE OR REPLACE TABLE statement for 'dim_transactions_flat' in the 'sprint3_gold' schema. It selects columns from 'fact_transactions_optimized' and 'products_clean' tables. The query is executed successfully, as indicated by a green checkmark and the message 'Se completó la consulta'. Below the message, it states 'Se está usando la cuota de procesamiento según demanda' and 'Ubicación de procesamiento: EU'.

```
1 CREATE OR REPLACE TABLE proverbial-deck-498507-c9.sprint3_gold.dim_transactions_flat AS
2 SELECT
3     t.transaction_id,
4     DATE (t.timestamp) AS transaction_date,
5     t.amount AS total_ticket,
6     p.product_id AS product_sku,
7     p.name AS product_name,
8     p.price AS product_price
9 FROM
10     proverbial-deck-498507-c9.sprint3_gold.fact_transactions_optimized AS t
11 CROSS JOIN UNNEST(t.product_ids) AS product_sku
12 INNER JOIN
13     proverbial-deck-498507-c9.sprint3_silver.products_clean AS p
14 ON product_sku = p.product_id;
```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda Ubicación de procesamiento: EU

Esquema de la taula dim_transactions_flat:



The screenshot shows a table schema view for 'dim_transactions_flat'. The interface includes a search bar, a 'Consulta' button, a 'Chat' button, and an 'Abrir' button. The 'Esquema' tab is selected, showing a table with 6 columns: transaction_id, transaction_date, total_ticket, product_sku, product_name, and product_price. Each column has a checkbox, a name, a type, a mode, and a description. A filter bar is also present above the table.

☐	Nombre del campo	Tipo	Modo	Descripción
☐	transaction_id	STRING	NULLABLE	-
☐	transaction_date	DATE	NULLABLE	-
☐	total_ticket	FLOAT	NULLABLE	-
☐	product_sku	INTEGER	NULLABLE	-
☐	product_name	STRING	NULLABLE	-
☐	product_price	FLOAT	NULLABLE	-

Exercici 2**Escenari:**

Ara que has preparat la infraestructura i disposes de la taula `dim_transactions_flat` (on cada fila equival a un producte venut), generar informes analítics és trivial. Ja no necessites funcions complexes de desanidament per respondre preguntes bàsiques de negoci.

Tasca:

Genera el Top 5 de productes més venuts en la història de la companyia.

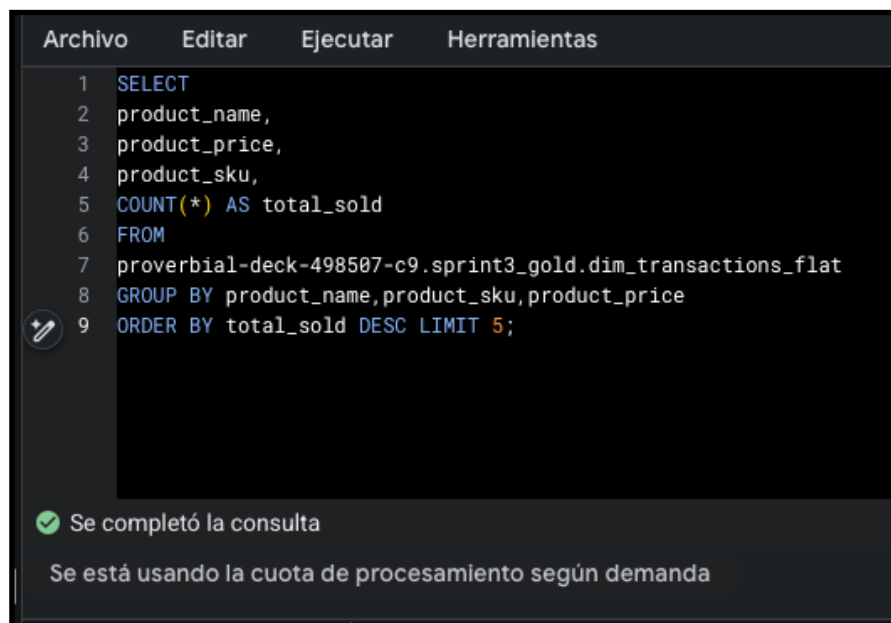
Mètode:

Consulta directament la teva nova taula desnormalitzada (`dim_transactions_flat`).

- En ser la taula plana, simplement necessites una agrupació estàndard (GROUP BY) pel nom del producte i un compte (COUNT) de les files.
- Entrega:
 - El codi SQL que genera el rànquing.
 - Una captura del resultat amb els 5 productes guanyadors i quantes unitats s'han venut de cadascun.

Resposta

Es mostra a continuació el codi SQL per a obtenir el TOP 5 de productes amb les dades bàsiques del producte:



```
Archivo  Editar  Ejecutar  Herramientas

1  SELECT
2  product_name,
3  product_price,
4  product_sku,
5  COUNT(*) AS total_sold
6  FROM
7  proverbial-deck-498507-c9.sprint3_gold.dim_transactions_flat
8  GROUP BY product_name,product_sku,product_price
9  ORDER BY total_sold DESC LIMIT 5;
```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Vista de la taula amb el TOP 5:

Recientes	Resultados de la consulta				
12:36 p.m.					
SELECT product_name,...					
	Fila	product_name	product_price	product_sku	total_sold
	1	riverlands the duel	31.52	52	2654
	2	Tully maester Tarly	167.2	29	2635
	3	duel Direwolf	96.9	21	2609
	4	the duel warden	180.91	16	2608
	5	mustafar jinn	2.12	66	2601

Exercici 3:

Escenari:

L'informe estàtic de l'exercici anterior ha agradat molt, però el Director de Màrqueting necessita professionalitzar-lo per a la presa de decisions diària. Té dos requeriments urgents:

1. Lògica Centralitzada: Volen veure el preu amb IVA (21%) desglossat. No obstant això, com que l'impost pot canviar per país o any, et prohibeixen posar el càlcul $\text{preu} * 1,21$ directament en el codi ("Hardcoded"). Has d'utilitzar una funció reutilitzable.
2. Frescor de Dades: El tauler ha d'estar actualitzat automàticament cada matí a les 07:00 AM, abans de la reunió diària de vendes.

Tasca:

Has d'evolucionar la teva taula `dim_transactions_flat` per incloure el càlcul d'impostos i automatitzar la seva regeneració.

Passos Tècnics:

1. User Defined Functions (UDF): Crea una funció SQL persistent anomenada `calculate_tax(amount)` que rebí un valor numèric i retorni el resultat aplicant el 21% d'impost.
2. Integració i Orquestració:
 - Modifica el codi de creació de la taula (de l'Exercici 1) perquè utilitzi la teva nova funció `calculate_tax` i generi una columna nova: `product_price_tax_inc`.
 - Configura una Scheduled Query a BigQuery perquè aquesta sentència (CTAS) s'executi automàticament cada dia a les 07:00 AM
3. Visualització (BI): Connecta Looker Studio a la teva taula `dim_transactions_flat` i crea un Dashboard: "Monitor de Rendiment de Vendes"

Hem creat la funció que rep el paràmetre amount de tipus FLOAT64 (decimal), aplica el factor d'impost multiplicant per 1.21 i retorna el resultat arrodonit a 2 decimals.

Aquesta funció es molt important perquè, si l'IVA canviés (per exemple, al 23%), només caldria modificar aquesta funció, totes les consultes i taules que la fan servir s'actualitzarien automàticament sense haver de buscar i canviar el valor a cada lloc on apareix.

```
1 CREATE OR REPLACE FUNCTION `proverbial-deck-498507-c9.sprint3_gold.calculate_tax`(amount FLOAT64)
2 RETURNS FLOAT64 AS (
3   ROUND(amount * 1.21, 2)
4 );
```

Esta consulta procesará 0 B cuando se ejecute.

```
1 CREATE OR REPLACE FUNCTION `proverbial-deck-498507-c9.sprint3_gold.calculate_tax`(amount FLOAT64)
2 RETURNS FLOAT64 AS (
3   ROUND(amount * 1.21, 2)
4 );
```

Se completó la consulta

Recientes	Resultados de la consulta	Chat	View all jobs	Guardar los resultados
9:23 p.m.	Información del trabajo	Resultados	Detalles de la ejecución	Gráfico de ejecución
CREATE OR REPLACE F...	La declaración creó una función nueva llamada proverbial-deck-498507-c9.sprint3_gold.calculate_tax.			

- sprint3_gold
 - Rutinas (1)
 - calculate_tax

Afegim al codi de l'exercici 1 la funció de l'IVA.

S'afegeix una sola línia al SELECT que crida la UDF passant-li p.price com a paràmetre. D'aquesta manera, aplica l'impost directament i si hi hagués qualsevol modificació, s'actualitzaria automàticament (un cop modificada la funció).

```

1 CREATE OR REPLACE TABLE sprint3_gold.dim_transactions_flat AS
2 SELECT
3   t.transaction_id,
4   DATE(t.timestamp) AS transaction_date,
5   t.amount AS total_ticket,
6   p.product_id AS product_sku,
7   p.name AS product_name,
8   p.price AS product_price,
9   sprint3_gold.calculate_tax(p.price) AS tax_included_price
10 FROM
11   sprint3_gold.fact_transactions_optimized AS t
12 CROSS JOIN UNNEST(t.product_ids) AS product_sku
13 INNER JOIN
14   sprint3_silver.products_clean AS p
15 ON product_sku = p.product_id;
16

```

✓ Esta consulta procesará 7.09 MB cuando se ejecute.
Se está usando la cuota de procesamiento según demanda

S'ha programat la taula dim_transactions_flat per regenerar-se automàticament cada dia a les 07:02 AM mitjançant una Scheduled Query de BigQuery.

S'ha escollit les 07:02 AM i no les 07:00 AM de forma intencionada, ja que, en entorns de producció, és habitual evitar les hores en punt (00, 30) perquè molts processos automatitzats s'executen en aquests moments, generant pics de càrrega als sistemes. D'aquesta manera, ens assegurem que l'equip de Màrqueting té la taula actualitzada abans de la seva reunió de vendes sense errades.

Configuración válida.

Detalles y programación

Nombre de la consulta programada *
Update dim_transactions_flat

Opciones de programación

Frecuencia de repetición *
Personalizado
Programa personalizado *
every day at 7:02am

☒ Comenzar ahora ☐ Comenzar a la hora definida

Fecha de inicio y hora de ejecución
16/6/26, 12:21 a.m.

☒ Sin hora de finalización programada
☐ Programar una hora de finalización

Fecha de finalización

Programada para ejecutarse en la programación personalizada
"every day at 7:02am" a partir de ahora.

Proposta de Dashboard:

"Monitor de Rendiment de Vendes"

Configura una pàgina amb 3 seccions clau:

Els "Big Numbers" (KPIs Generals): A la part superior, col·loca tres targetes de resultats (Scorecards) per donar context immediat.

KPI 1: Ingressos Totals (Amb IVA)

- Mètrica: $\text{SUM}(\text{product_price_tax_inc})$, per què: Mostra el volum real de diners que entra.

Transaccions Úniques (Comandes)

- Mètrica: $\text{COUNT_DISTINCT}(\text{transaction_id})$, Transaccions Úniques (Comandes)

Camp Calculat:

Crea un camp a Looker: $\text{SUM}(\text{product_price_tax_inc}) / \text{COUNT_DISTINCT}(\text{transaction_id})$

2. Evolució Temporal (Gràfic de Sèries Temporals) Sota els KPIs, un gràfic de línia que ocupi tot l'ample.

- Dimensió: timestamp (Desglossat per Data).
- Mètrica: $\text{SUM}(\text{product_price_tax_inc})$.
- Línia de Referència (Opcional): Afegeix una mitjana constant o una línia de tendència.
- Valor: Permet a Màrqueting veure si les vendes pugen o baixen dia a dia, o si hi va haver un pic per alguna campanya específica.

El "Top Productes" Enriquit (Taula amb Mapa de Calor) Oblida't del gràfic de barres simple. Utilitza una Taula amb Barres incrustades o Mapa de Calor.

- Dimensió: product_name.
- Mètrica 1: $\text{COUNT}(\ast) \rightarrow$ Renombra-ho a "Unitats Venudes".
- Mètrica 2: $\text{SUM}(\text{product_price_tax_inc}) \rightarrow$ Renombra-ho a "Ingressos Totals".
- Mètrica 3: $\text{SUM}(\text{product_price_tax_inc}) - \text{SUM}(\text{product_unit_price}) \rightarrow$ Renombra-ho a "IVA Recaptat" (Això demostra el valor de la teva UDF).
- Estil: A la configuració d'estil de la taula, activa l'opció "Mostra número com a: Barra" o "Mapa de calor" per a la columna d'Ingressos.

Resposta

A continuació trobem l'enllaç del Dashboard creat a Looker Studio:

<https://datastudio.google.com/s/t67dEvkiXcQ>

El dashboard conecta directament amb la taula `dim_transactions_flat` de BigQuery i es divideix en tres seccions:

KPIs generals: tres targetes a la part superior que mostren:

- els ingressos totals amb IVA (`SUM(product_price_tax_inc)`)
- el nombre de comandes úniques (`COUNT_DISTINCT(transaction_id)`)
- el tiquet mitjà per transacció (camp calculat: `SUM(product_price_tax_inc) / COUNT_DISTINCT(transaction_id)`)

Evolució temporal :un gràfic de línies amb `transaction_date` com a dimensió i `SUM(product_price_tax_inc)` com a mètrica, que mostra com evolucionen els ingressos dia a dia, permetent detectar pics de vendes o caigudes relacionades amb campanyes de màrqueting. Hem afegit també la mijta del total gastat. Obtenim de forma molt visual les dades per dies.

Top Productes :una taula amb mapa de calor que mostra, per a cada `product_name`, les unitats venudes (`COUNT(*)`), els ingressos totals (`SUM(product_price_tax_inc)`) i l'IVA recaptat (camp calculat: `SUM(product_price_tax_inc) - SUM(product_price)`), deixant visible d'un sol cop quin producte genera més valor per al negoci i poder treure'n conclusions o bé prendre decisions d'una forma més visual.

El dashboard centralitza en una sola pantalla tota la informació clau per a la presa de decisions diària: els ingressos totals amb IVA, el nombre de comandes, el tiquet mitjà, l'evolució temporal de les vendes i el rànding de productes mes venuts.

Gràcies a la connexió directa amb `dim_transactions_flat` i la regeneració automàtica de les 07:02 AM, les dades sempre estan actualitzades sense cap intervenció manual, permetent a l'equip de Màrqueting i a la Direcció detectar tendències, pics de vendes o caigudes de rendiment en temps real, abans de començar la jornada laboral.